I-Logical Approaches Knowledge Bases, Entailment and Contractions of Tensors

Foundations of Neuro-Symbolic AI

Alex Goessmann

University of Applied Science Würzburg-Schweinfurt (THWS)

March 23, 2026

Outline

- Knowledge Bases
- Tensor Networks
- Entailment
- Python package `tnreason`

Recall: Curse of Dimensionality

Observation

The number of coordinates of a propositional formula increases exponentially with the number d of variables.

Its not just a depiction problem - it is a fundamental intractability in storage and processing!

Curse of dimensionality: Storage problem

When reasoning about a factored system with many variables (i.e. d is large), the naive enumeration of all models to formulas is infeasible.

Motivation: Knowledge Base Representation

There is further sparsity in the

- We often have a list of formulas.
- Each formula is just affected by a few variables.

Definition

Knowledge base Knowledge bases are conjunctions over sets of formulas $\{f_\ell : \ell \in [p]\}$

$$\mathcal{KB} = \bigwedge_{\ell \in [p]} f_\ell$$

How to exploit this sparsity in the effective representation of our knowledge?

Outline

- Knowledge Bases
- Tensor Networks
- Entailment
- Python package `tnreason`

Separated Combination of Tensors: Tensor Products

Definition (Tensor Product)

Let there be two tensor

$$\tau\left[X_{[d]}\right] \;:\; \bigtimes_{k\in[d]} [m_k] \to \mathbb{R} \quad \text{and} \quad \tilde{\tau}\left[Y_{[p]}\right] \;:\; \bigtimes_{\ell\in[p]} [n_\ell] \to \mathbb{R}\,.$$

Then there tensor product is the map

$$(\tau \otimes \tilde{\tau})\left[X_{[d]}, Y_{[p]}\right] : \left(\bigtimes_{k\in[d]} [m_k]\right) \times \left(\bigtimes_{\ell\in[p]} [n_\ell]\right) \to \mathbb{R}$$

defined for $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ and $y_{[p]} \in \bigtimes_{\ell\in[p]}[n_\ell]$ as

$$(\tau \otimes \tilde{\tau})\left[X_{[d]} = x_{[d]}, Y_{[p]} = y_{[p]}\right] = \tau\left[X_{[d]} = x_{[d]}\right] \cdot \tilde{\tau}\left[Y_{[p]} = y_{[p]}\right]\,.$$

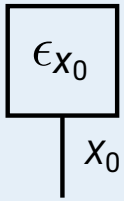
Example: One-hot encodings

Definition (One-hot encoding to atomic representations)

The one-hot encoding of a variable X_0 with dimension $m_0 \in \mathbb{N}$ is the map

$$\epsilon_{x_0}\left[X_0 = y_0\right] = \begin{cases} 1 & \text{if } x_0 = y_0 \\ 0 & \text{else} \end{cases}$$

that is the vector in $\mathbb{R}^{m_0}$ with a 1 at the position of the state and 0 else.



$$= \begin{array}{c} \begin{pmatrix} 0 & \cdots & 0 & 1 & 0 & \cdots & 0 \end{pmatrix} \\ \begin{array}{ccccccc} \vdots & & & \vdots & & & \vdots \\ 0 & & & x_0 & & & m_0 - 1 \end{array} \end{array} \left[X_0\right]$$

Example: The one-hot encoding $\epsilon_2\left[X_0\right] \in \mathbb{R}^4$ has the coordinates

$$\epsilon_2\left[X_0\right] = \begin{pmatrix} 0 & 0 & 1 & 0 \end{pmatrix}\left[X_0\right]$$

Example: One-hot encodings

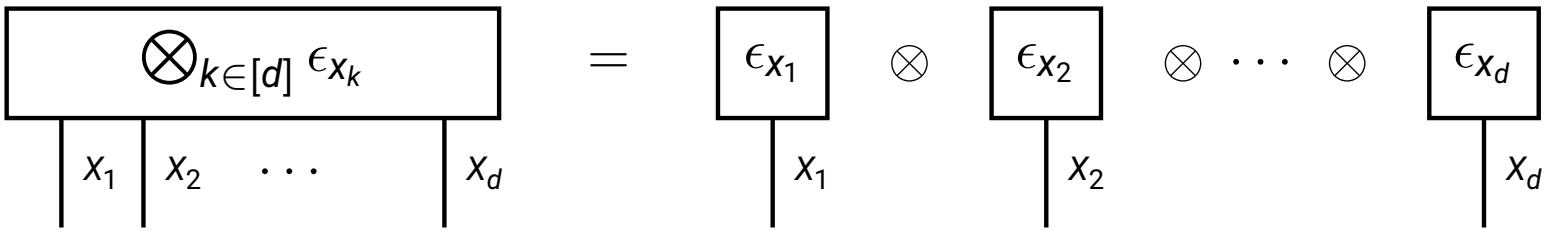
Definition (One-hot encoding to factored representations)

The one-hot encoding of a factored representation with variables $X_{[d]} = \{X_k \ : , k \in [d]\}$ in state $x_0, \ldots, x_{d-1}$ is the tensor product

$$\epsilon_{x_{[d]}} \left[X_{[d]}\right] = \bigotimes_{k \in [d]} \epsilon_{x_k} \left[X_k\right] \tag{1}$$

of the one-hot encodings of the variables.

We can depict the one-hot encoding by



Example: One-hot encodings

Example: The one-hot encoding $\epsilon_{(0,2,1)}\left[X_0, X_1, X_2\right] \in \bigotimes_{k\in[3]} \mathbb{R}^4$ with the coordinates

$$\epsilon_{(0,1,1)}\left[X_0 = x_0, X_1 = x_1, X_2 = x_2\right] = \begin{cases} 1 & \text{if } x_0 = 0,\ x_1 = 1 \text{ and } x_2 = 1 \\ 0 & \text{else} \end{cases}$$

is the tensor product of

$$\epsilon_{(0)}(x_0) = \begin{cases} 1 & \text{if } x_0 = 0 \\ 0 & \text{else} \end{cases} \quad , \quad \epsilon_{(2)}(x_1) = \begin{cases} 1 & \text{if } x_1 = 1 \\ 0 & \text{else} \end{cases}$$

and

$$\epsilon_{(1)}(x_2) = \begin{cases} 1 & \text{if } x_2 = 1 \\ 0 & \text{else} \end{cases} .$$

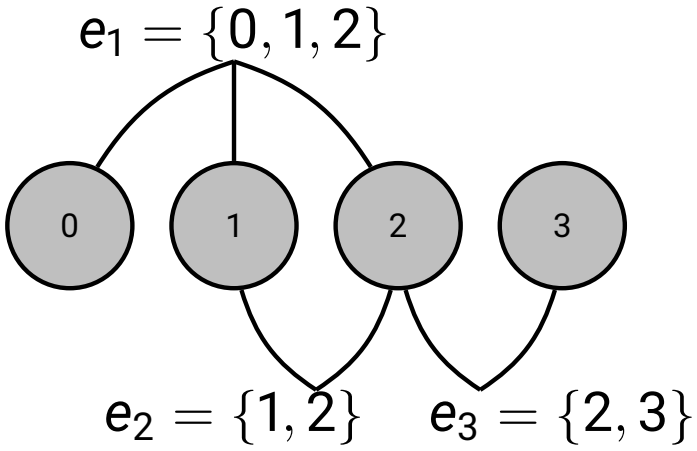
Depicting common legs: Hypergraphs

Definition

Hypergraph A hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of two sets:

- Nodes $v \in \mathcal{V}$
- Hyperedges $e \in \mathcal{E}$, where each is is a subset $e \subset \mathcal{V}$ of nodes

We depict nodes by gray circles and hyperedges as connections of them:



In this example:

- $\mathcal{V} = \{0, 1, 2, 3\}$
- $\mathcal{E} = \{\{0, 1, 2\}, \{1, 2\}, \{2, 3\}\}$

Associating tensors with hyperedges

Let us use the hypergraph formalism to depict tensor networks

- Nodes are decorated by the categorical variables on the tensor legs:

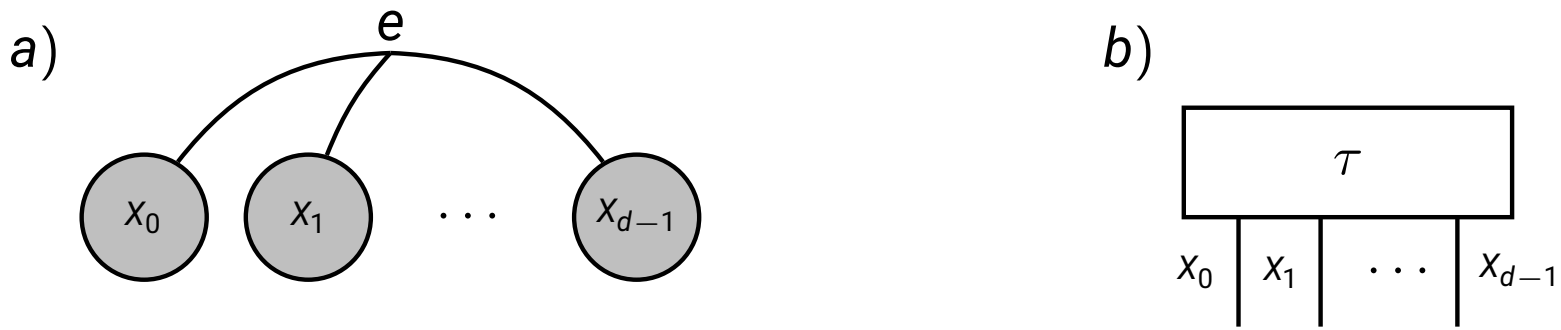
$$\{0, \ldots, d-1\} \quad \text{represent the variables} \quad \{X_0, \ldots, X_{d-1}\}$$

  Further, we associate the dimensions m_v of the respective variable with them.

- Hyperedges are decorated by tensors

$$e = \{v \,:\, v \in e\} \quad \text{represents the tensor} \quad \tau^e\,[X_e] \in \bigotimes_{v \in e} \mathbb{R}^{m_v}$$

We now decorate hyperedges by tensors



Tensor Networks

Definition (Tensor Network)

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a hypergraph (that is edges are arbitrary nonempty subsets of nodes) with nodes decorated by random variables X_v with dimensions

$$m_v \in \mathbb{N}$$

and hyperedges $e \in \mathcal{E}$ decorated by core tensors

$$\tau^e\left[X_e\right] \in \bigotimes_{v \in e} \mathbb{R}^{m_v} .$$

Then we call the set

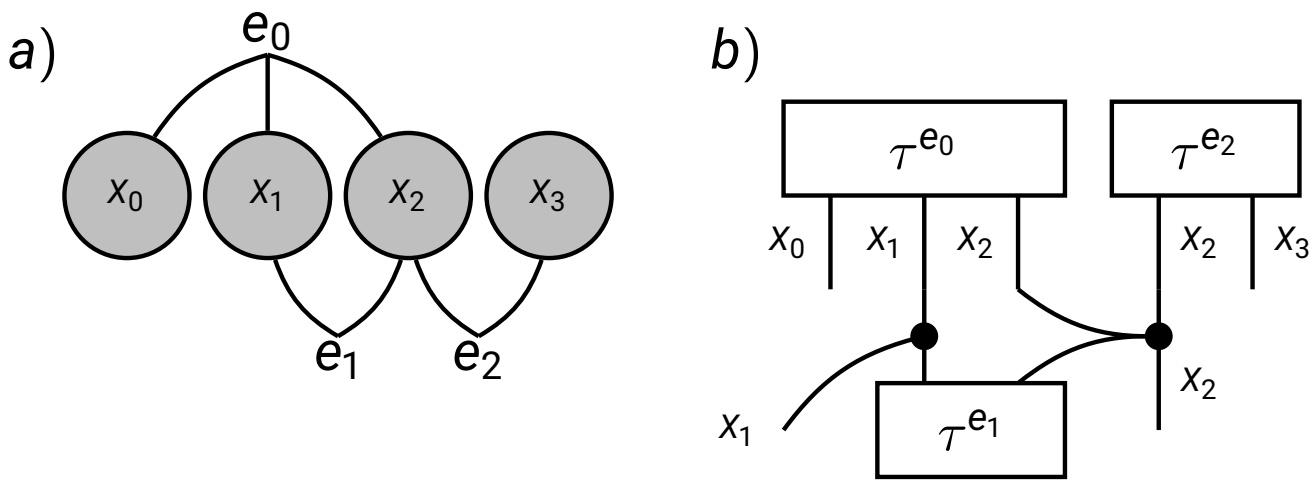
$$\tau^{\mathcal{G}}\left[X_{\mathcal{V}}\right] := \left\{\tau^e\left[X_e\right] \,:\, e \in \mathcal{E}\right\}$$

the tensor network to the decorated hypergraph $\mathcal{G}$.

Graphical Representation of Tensor Networks

Example of a tensor network:

- Four categorical variables $\mathcal{V} = \{X_0, X_1, X_2, X_3\}$ appearing as nodes and decorated by their dimensions $m_{:}$.
- Hyperedges $e_0 = \{X_0, X_1, X_2\}$, $e_1 = \{X_1, X_2\}$ and $e_2 = \{X_2, X_3\}$ decorated with tensors



We choose two diagrammatic depictions:

a) The defining Hypergraph
b) The contraction diagram

Contraction of Tensor Networks

Definition (Contraction)

Let $\tau^{\mathcal{G}}$ be a tensor network on a decorated hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. For any subset $\mathcal{U} \subset \mathcal{V}$ we define the contraction to be the tensor

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}]} \in \bigotimes_{v \in \mathcal{U}} \mathbb{R}^{m_v} \tag{2}$$

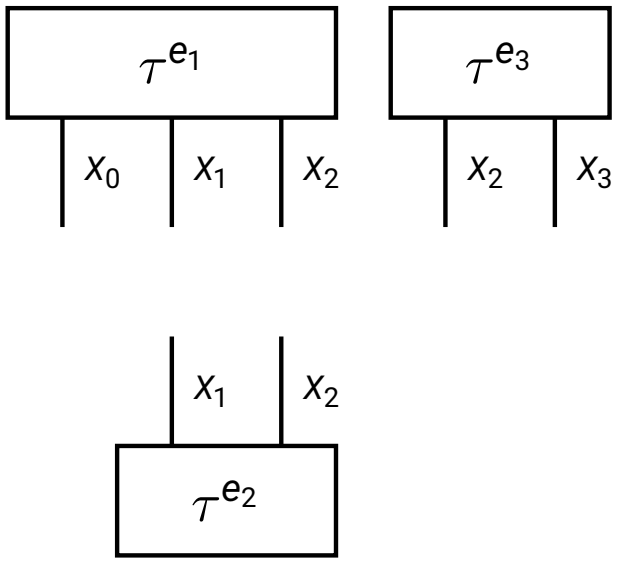
defined coordinatewise by the sum

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}} = x_{\mathcal{U}}]} = \sum_{\{x_v \in m_v \,:\, v \in \mathcal{V}/\mathcal{U}\}} \prod_{e \in \mathcal{E}} \tau^e \left[X_e = x_e \right] . \tag{3}$$

Tensor Networks

Tensor Networks are build by

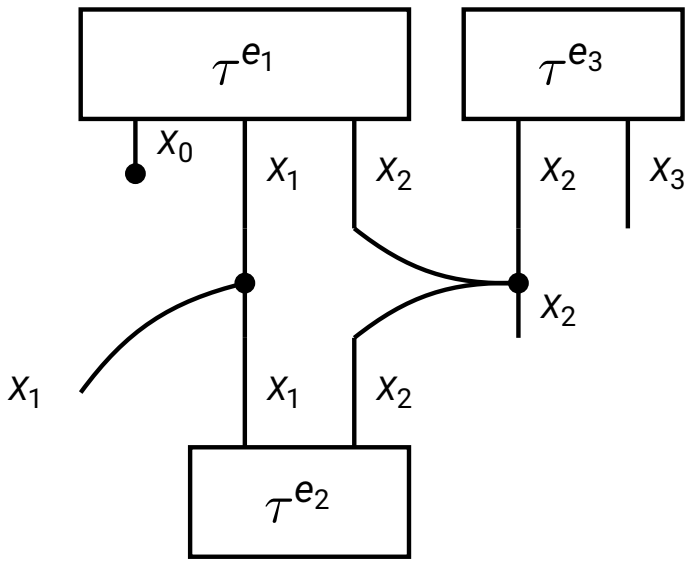
- Separated Combination of Tensors $\tau^{e_1}, \tau^{e_2}, \tau^{e_3}$: Tensor Products



Tensor Networks

Tensor Networks are build by

- Separated Combination of Tensors $\tau^{e_1}, \tau^{e_2}, \tau^{e_3}$: Tensor Products
- Connecting Tensors along common legs and closing legs: Contractions



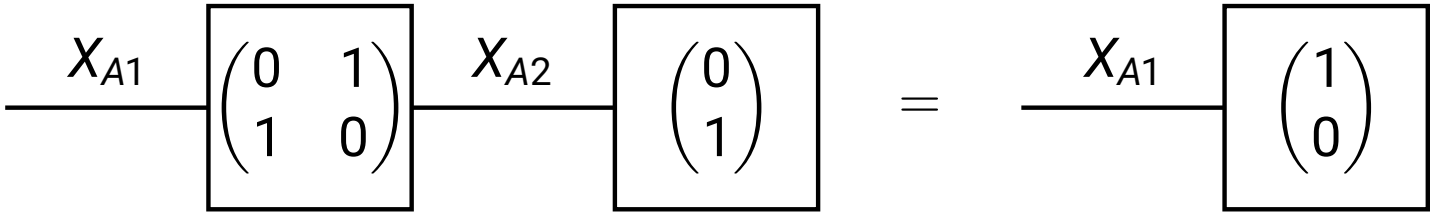
Example: Matrix-Vector Multiplications

The blockwise notation is especially useful in visualizing contractions. The simplest contraction is the matrix-vector multiplications.

Example: Knowing that $X_{A2} = 1$, what are the possible values of X_{A1}?

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} [X_{A1}, X_{A2}] \begin{pmatrix} 0 \\ 1 \end{pmatrix} [X_{A2}] \quad = \quad \begin{pmatrix} 1 \\ 0 \end{pmatrix} [X_{A1}]$$

In the blockwise notation the matrix-vector multiplication is depicted by:



Example of a Tensor Contraction: Hadamard Product

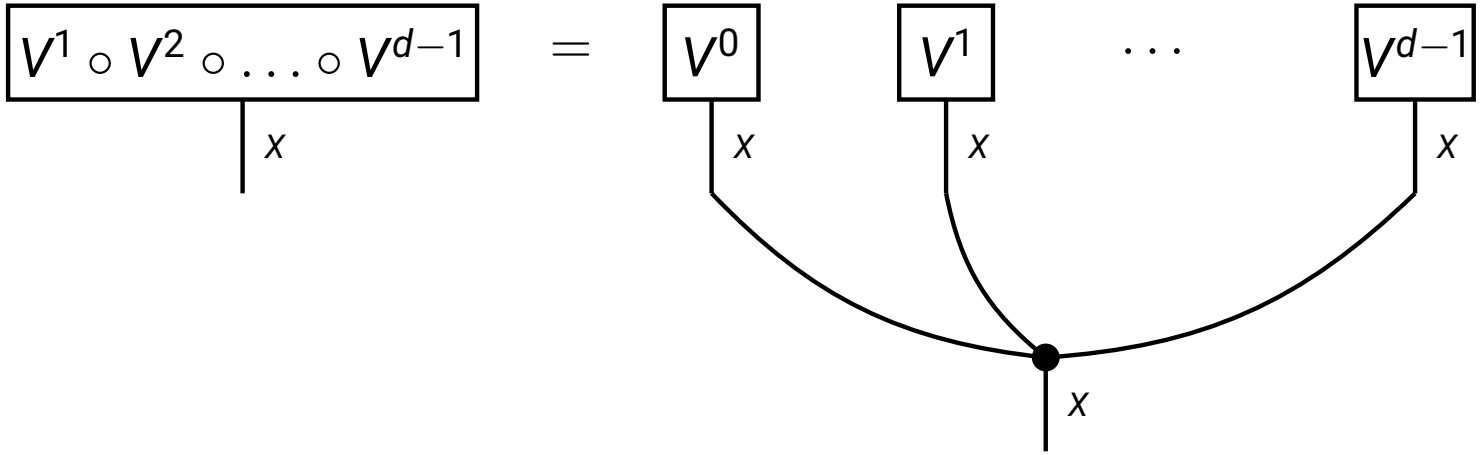
Let $V^k \in \mathbb{R}^p$ be vectors for $k \in [d]$. Their Hadamard product is the vector

$$V^1 \circ V^2 \circ \ldots \circ V^{d-1} \in \mathbb{R}^p \tag{4}$$

defined by

$$\left(V^1 \circ V^2 \circ \ldots \circ V^{d-1}\right)_x = \prod_{k \in [d]} V^k_x \,. \tag{5}$$

We visualize the product by leg connection of multiple vectors:



Curse of dimensionality of factored representations

Tensor spaces have a problem with exponential increase of dimensions as

$$\dim\left[\bigotimes_{v\in\mathcal{V}}\mathbb{R}^{m_v}\right]=\prod_{v\in\mathcal{V}}m_v\,.$$

Tensor Networks can break this curse of dimensionality by storing the decorating tensors instead

$$\sum_{e\in\mathcal{E}}\dim\left[\bigotimes_{v\in e}\mathbb{R}^{m_v}\right]=\sum_{e\in\mathcal{E}}\left(\prod_{v\in e}m_v\right)$$

Representation strategy

Store the recipe (that is the core tensors), not the result (the contracted tensor network).

Knowledge Bases as Contractions

We model each formula in a Knowledge base as a decoration

- hyperedge e_ℓ of affected variables
- decorating tensor $f_\ell\left[X_{\mathsf{e}_\ell}\right]$

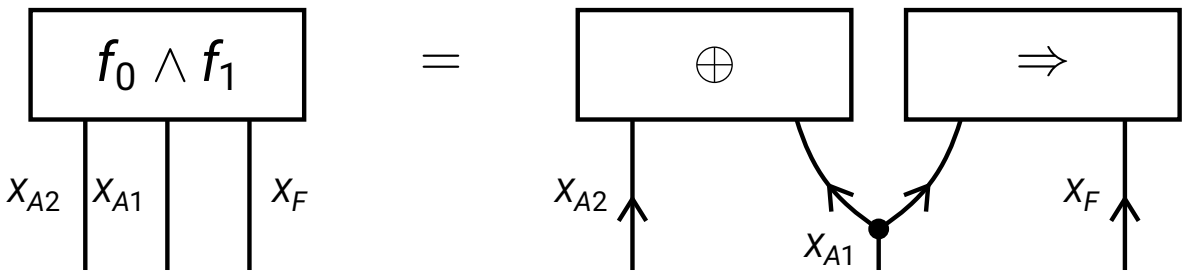
For any Knowledge base we have

$$\mathcal{KB}\left[X_{\mathcal{V}}\right] = \left\langle f_\ell\left[X_{\mathsf{e}_\ell}\right]\right\rangle_{\left[X_{\mathcal{V}}\right]}$$

Tensor network decomposition of the toy accounting system

Toy accounting system:

- $f_0 = X_{A1} \oplus X_{A2}$ by $e_0 = \{A1, A2\}$
- $f_1 = X_F \Rightarrow X_{A1}$ by $e_1 = \{F, A1\}$



Knowledge bases are decomposed into a tensor network of formulas.

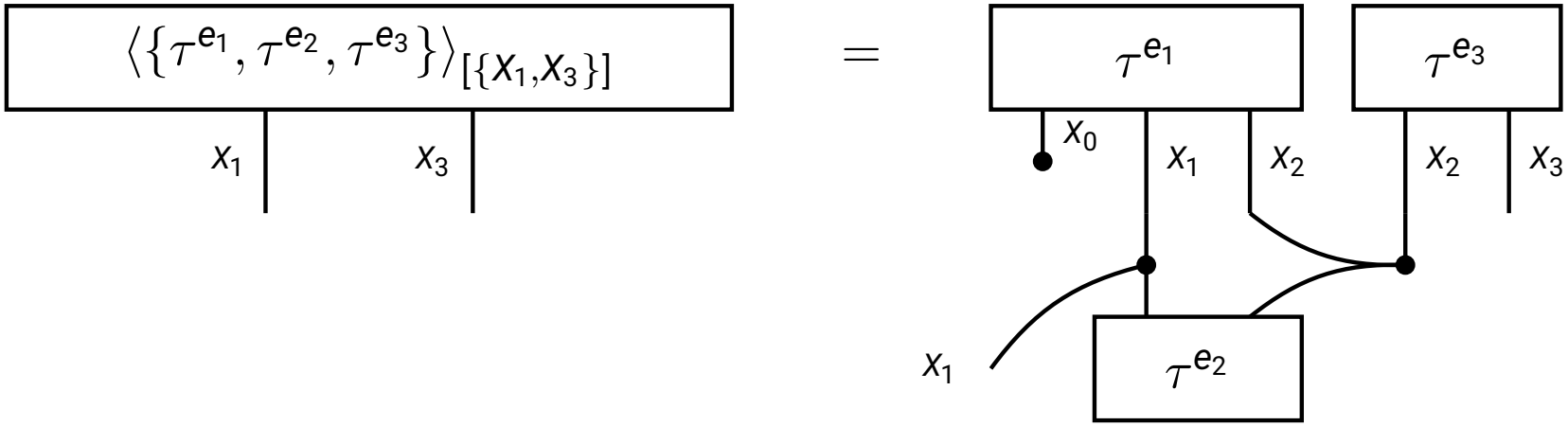
Outline

- Knowledge Bases
- Tensor Networks
- Entailment
- Python package `tnreason`

Contractions with closed variables

Example of a contraction:

- Four categorical variables $\mathcal{V} = \{X_0, X_1, X_2, X_3\}$ appearing as nodes and decorated by their dimensions $m_{:}$.
- Hyperedges $e_0 = \{X_0, X_1, X_2\}$, $e_1 = \{X_1, X_2\}$ and $e_2 = \{X_2, X_3\}$ decorated with tensors
- Contraction leaving $\{X_1, X_3\}$ open, by summing over the states of $\{X_0, X_2\}$



Model counts

So far: Knowledge bases as contractions with all variables open. Now: Contractions without open variables.
We have

$$\begin{aligned}&\left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]}\\ &\quad = \#\{x_{[d]} \in [2]^d \,:\, \mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = f\left[X_{[d]} = x_{[d]}\right] = 1\}\end{aligned}$$

That is, the number of common models is counted
So if

$$\left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \left\langle \mathcal{KB}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}$$

then $\mathcal{KB} \models f$.

Entailment

Definition (Entailment)

Let $\mathcal{KB}\left[X_{[d]}\right]$ and $f\left[X_{[d]}\right]$ be two propositional formulas. If for all worlds $x_{[d]}$, for which $\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1$, we also have $f\left[X_{[d]} = x_{[d]}\right] = 1$, then we say that f is entailed by $\mathcal{KB}$ and denote

$$\mathcal{KB} \models f\,.$$

Aim: How to express this as a computation?

We develop the contraction formalism of tensors for this.

Entailment by contractions

Trick:

$$\left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \left\langle \mathcal{KB}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}$$

is equal to

$$\left\langle \mathcal{KB}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} - \left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = 0$$

and

$$\left\langle \mathcal{KB}\left[X_{[d]}\right], \neg f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = 0$$

Contraction criterion for entailment

For any propositional formulas we have $\mathcal{KB} \models f$ if and only if

$$\left\langle \mathcal{KB}\left[X_{[d]}\right], \neg f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = 0$$

Outline

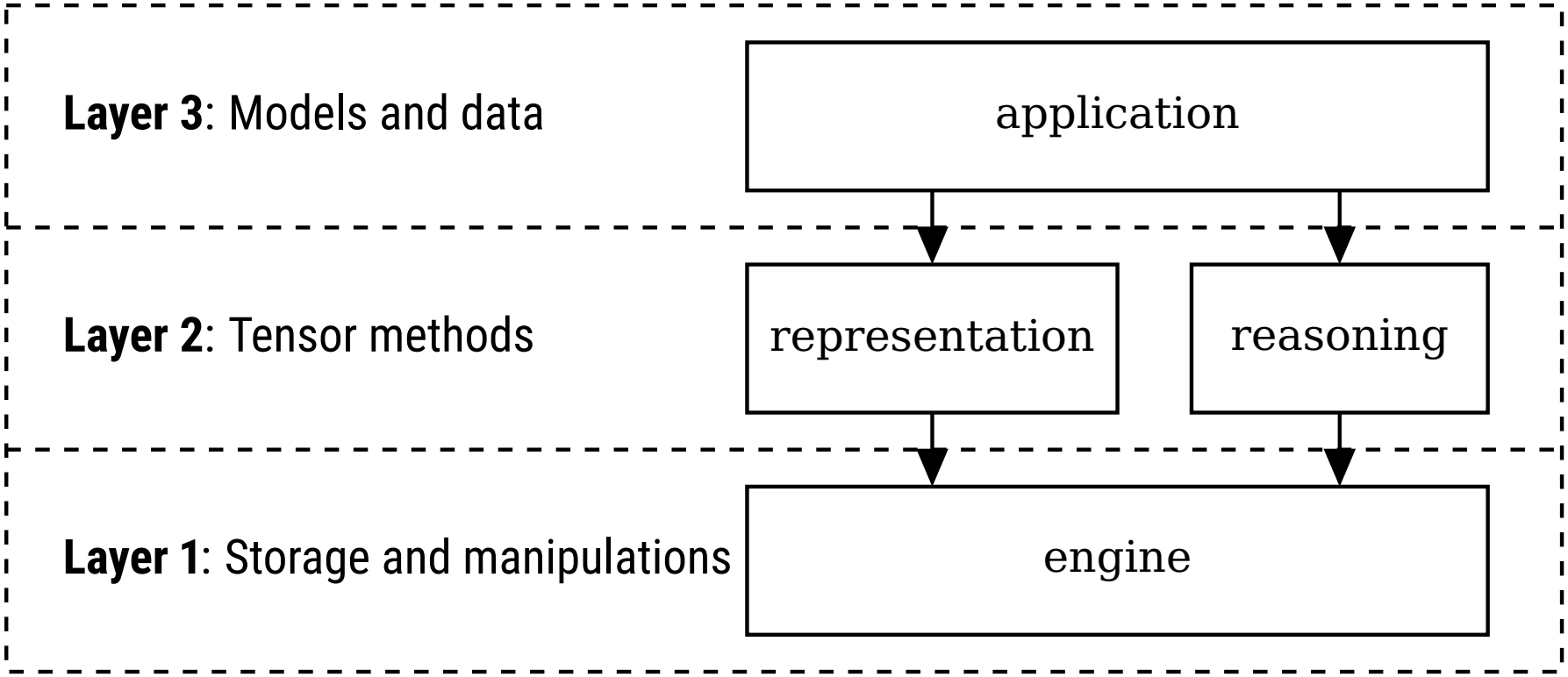
- Knowledge Bases
- Tensor Networks
- Entailment
- Python package `tnreason`

Python package tnreason

tnreason = Tensor-Network Reasoning

Functionality:

- Representations of logical and probabilistic models as tensor networks
- Neuro-symbolic reasoning based on tensor network contractions



Installation

tnreasonis maintained on github:

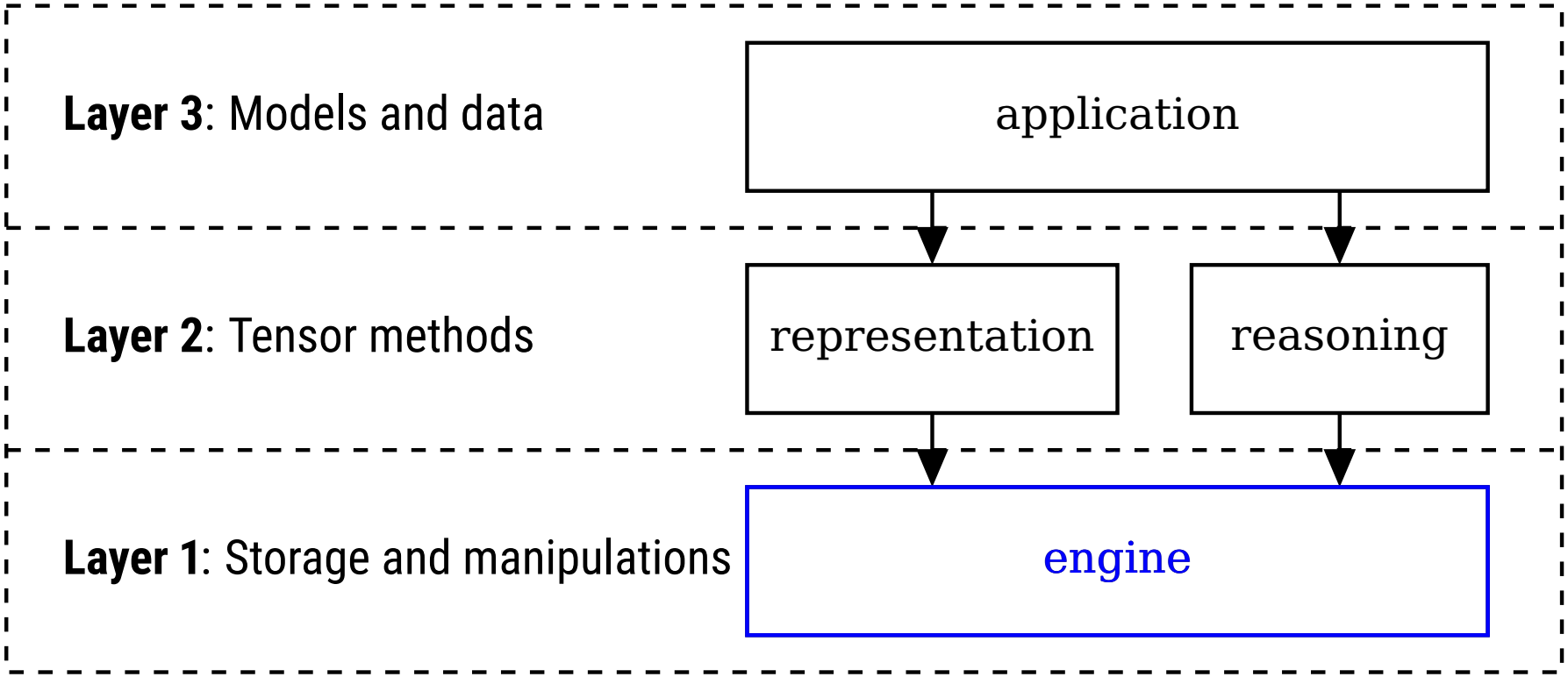
https://github.com/tnreason/tnreason-py

Installation using pip:

```
pip install tnreason==2.0.0
```

Implementation in tnreason: Subpackage engine

The subpackage engineis dedicated to tensor networks and contractions.



Subpackage engine: Cores

Each Tensor core has attributes

- values (array-like): storing the value of the coordinates
- colors (list of str): specifying the name of the variables represented by its axes
- name (str): to distinguish from other cores

The implemented core types differ in the values argument. Cores are instantiated by

```
from tnreason import engine
engine.getCore(coreType)(coreValues, coreColors,
↪  coreName)
```

Subpackage engine: Contractions

Reflected in the notation

$$\langle \tau^{\mathcal{G}} \rangle_{[\mathcal{V}]}$$

a contraction is defined by

- Tensor Network $\tau^{\mathcal{G}}$, i.e. a dictionary of tensor cores
- Open Variables $\mathcal{V}$

Contraction calls are done by

```
from tnreason import engine
engine.contract(coreDict, openColors)
```

Where

- coreDict: Dictionary of tensor cores (of the above formats), representing the Tensor Network $\tau^{\mathcal{G}}$
- openColors: List $\mathcal{V}$ of str, each str identifying a color, that is a variable to be left open in the contraction